JobSheet - Week 4 - Polymorphism

Nama: Darva Aryasatya Putra Hermawan

NIM: 251524005

Kelas: 1A

Repo GitHub: https://github.com/Teknik-Pemrograman-Praktek/Latihan-1A---Week-4

No. 1 Inheritance via Polymorphism (Another Type of Employee)

Soal Praktikum
Silahkan membaca soal pada file lampiran: 1-Polymorphism via Inheritance.pdf
Source Code

Commission.java

```
public class Commission extends Hourly{
   private double totalSales;
   private double commissionRate;

    public Commission(String eName, String eAddress, String ePhone, String
socSecNumber, double rate, double commissionRate) {
       super (eName, eAddress, ePhone, socSecNumber, rate);
       totalSales = 0.0;
       this.commissionRate = commissionRate;
   }

   public void addSales(double totalSales) {
       this.totalSales += totalSales;
   }

   @Override
   public double pay(){
       double payment = super.pay() + (totalSales * commissionRate);
       totalSales = 0.0;
       return payment;
   }

   @Override
   public String toString() {
       String result = super.toString();
       result += "\nTotal Sales: " + totalSales;
       return result;
   }
}
```

Employee.java

```
public class Employee extends StaffMember
{
   protected String socialSecurityNumber;
   protected double payRate;
   //-----------------------------------------------------------------
```

```java
   // Sets up an employee with the specified information.
   //-----------------------------------------------------------------
   public Employee (String eName, String eAddress, String ePhone,
                    String socSecNumber, double rate)
   {
       super (eName, eAddress, ePhone);
       socialSecurityNumber = socSecNumber;
       payRate = rate;
   }
   //-----------------------------------------------------------------
   // Returns information about an employee as a string.
   //-----------------------------------------------------------------
   @Override
   public String toString()
   {
       String result = super.toString();
       result += "\nSocial Security Number: " + socialSecurityNumber;
       return result;
   }
   //-----------------------------------------------------------------
   // Returns the pay rate for this employee.
   //-----------------------------------------------------------------
   @Override
   public double pay()
   {
       return payRate;
   }
}
```

Executive.java

```java
public class Executive extends Employee
{
   private double bonus;
   //-----------------------------------------------------------------
// Sets up an executive with the specified information.
//-----------------------------------------------------------------
    public Executive (String eName, String eAddress, String ePhone, String
socSecNumber, double rate)
   {
       super (eName, eAddress, ePhone, socSecNumber, rate);
       bonus = 0; // bonus has yet to be awarded
   }
   //-----------------------------------------------------------------
// Awards the specified bonus to this executive.
//-----------------------------------------------------------------
   public void awardBonus (double execBonus)
   {
       bonus = execBonus;
   }
   //-----------------------------------------------------------------
// Computes and returns the pay for an executive, which is the
// regular employee payment plus a one-time bonus.
//-----------------------------------------------------------------
   @Override
   public double pay()
   {
       double payment = super.pay() + bonus;
       bonus = 0;
       return payment;
   }
}
```

Firm.java

```
public class Firm
{
   //-----------------------------------------------------------------
// Creates a staff of employees for a firm and pays them.
//-----------------------------------------------------------------
   public static void main (String[] args)
   {
       Staff personnel = new Staff();
       personnel.payday();
   }
}
```

Hourly.java

```
public class Hourly extends Employee
{
   private int hoursWorked;
   //-----------------------------------------------------------------
   // Sets up this hourly employee using the specified information.
   //-----------------------------------------------------------------
   public Hourly (String eName, String eAddress, String ePhone,
                  String socSecNumber, double rate)
   {
       super (eName, eAddress, ePhone, socSecNumber, rate);
       hoursWorked = 0;
   }
   //-----------------------------------------------------------------
   // Adds the specified number of hours to this employee's
   // accumulated hours.
   //-----------------------------------------------------------------
   public void addHours (int moreHours)
   {
       hoursWorked += moreHours;
   }
   //-----------------------------------------------------------------
   // Computes and returns the pay for this hourly employee.
   //-----------------------------------------------------------------
   @Override
   public double pay()
   {
       double payment = payRate * hoursWorked;
       hoursWorked = 0;
       return payment;
   }
   //-----------------------------------------------------------------
   // Returns information about this hourly employee as a string.
   //-----------------------------------------------------------------
   @Override
   public String toString()
   {
       String result = super.toString();
       result += "\nCurrent hours: " + hoursWorked;
       return result;
   }
}
```

Staff.java

```
public class Staff
{
   StaffMember[] staffList;
   //-----------------------------------------------------------------
// Sets up the list of staff members.
```

```
//-----------------------------------------------------------------
   public Staff ()
   {
       staffList = new StaffMember[8];
         staffList[0] = new Executive ("Sam", "123 Main Line", "555-0469",
"123-45-6789", 2423.07);
         staffList[1] = new Employee ("Carla", "456 Off Line", "555-0101",
"987-65-4321", 1246.15);
        staffList[2] = new Employee ("Woody", "789 Off Rocker", "555-0000",
"010-20-3040", 1169.23);
         staffList[3] = new Hourly ("Diane", "678 Fifth Ave.", "555-0690",
"958-47-3625", 10.55);
       staffList[4] = new Volunteer ("Norm", "987 Suds Blvd.", "555-8374");
       staffList[5] = new Volunteer ("Cliff", "321 Duds Lane", "555-7282");
           staffList[6] = new Commission ("Darva", "Politeknik Bandung",
"555-8696", "958-47-3666", 6.25, 0.20);
             staffList[7] = new Commission ("Arva", "ITB", "555-8626",
"958-47-3266", 9.75, 0.15);
       ((Executive)staffList[0]).awardBonus (500.00);
       ((Hourly)staffList[3]).addHours (40);
       ((Commission)staffList[6]).addHours(35);
       ((Commission)staffList[6]).addSales(400);
       ((Commission)staffList[7]).addHours(40);
       ((Commission)staffList[7]).addSales(950);
   }
   //-----------------------------------------------------------------
// Pays all staff members.
//-----------------------------------------------------------------
   public void payday ()
   {
       double amount;
       for (int count=0; count < staffList.length; count++)
       {
           System.out.println (staffList[count]);
           amount = staffList[count].pay(); // polymorphic
           if (amount == 0.0)
               System.out.println ("Thanks!");
           else
               System.out.println ("Paid: " + amount);
           System.out.println ("-----------------------------------");
       }
   }
}
```

StaffMember.java

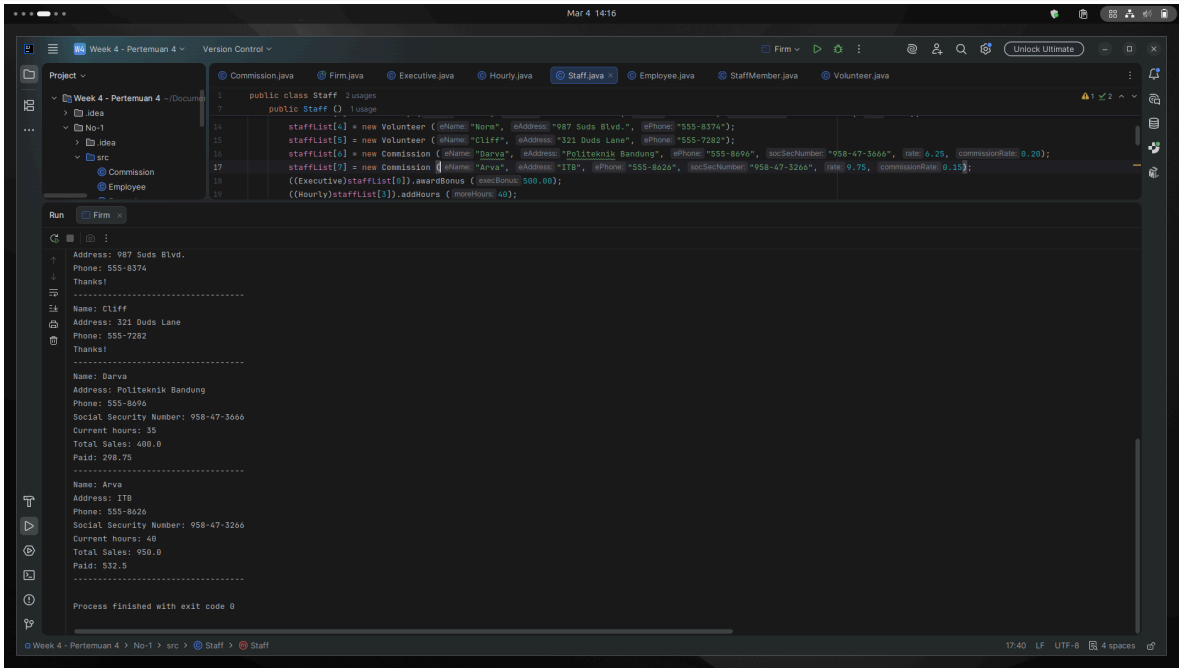
```
abstract public class StaffMember
{
   protected String name;
   protected String address;
   protected String phone;
   //-----------------------------------------------------------------
   // Sets up a staff member using the specified information.
   //-----------------------------------------------------------------
   public StaffMember (String eName, String eAddress, String ePhone)
   {
       name = eName;
       address = eAddress;
       phone = ePhone;
   }
   //-----------------------------------------------------------------
   // Returns a string including the basic employee information.
   //-----------------------------------------------------------------
```

```java
    public String toString()
    {
        String result = "Name: " + name + "\n";
        result += "Address: " + address + "\n";
        result += "Phone: " + phone;
        return result;
    }
    //-----------------------------------------------------------------
    // Derived classes must define the pay method for each type of
    // employee.
    //-----------------------------------------------------------------
    public abstract double pay();
}
```

Volunteer.java

```java
public class Volunteer extends StaffMember
{
    //-----------------------------------------------------------------
// Sets up a volunteer using the specified information.
//-----------------------------------------------------------------
    public Volunteer (String eName, String eAddress, String ePhone)
    {
        super (eName, eAddress, ePhone);
    }
    //-----------------------------------------------------------------
// Returns a zero pay value for this volunteer.
//-----------------------------------------------------------------
    public double pay()
    {
        return 0.0;
    }
}
```

Screenshot Hasil



Penjelasan Permasalahan dan Solusi

Permasalahan:

Perlu menambahkan tipe karyawan baru bernama Commission yaitu karyawan hourly yang juga mendapat komisi dari penjualan. Class ini belum ada di hierarki yang sudah tersedia, jadi harus di tambahkan.

Solusi:

Membuat class Commission extends Hourly dengan dua variable baru: totalSales dan commissionRate. Constructor menerima 6 parameter, 5 diteruskan ke Hourly via super(), sisanya untuk commissionRate. totalSales diinisialisasi 0.0 karena belum ada penjualan saat objek dibuat.

Override pay(): pay() di override agar menghitung gaji jam kerja via super.pay() lalu ditambah komisi (totalSales * commissionRate). totalSales direset ke 0 setelahnya. lalu hoursWorked tidak perlu direset karena sudah otomatis direset di dalam Hourly.pay().

Override toString(): toString() di override untuk menambahkan info totalSales ke output dari super.toString().

Staff.java diperbarui ukuran array diperbesar ke 8, ditambah dua objek Commission lengkap dengan addHours() dan addSales().

Nama Teman Hal yang Dibantu (Opsional)

No. 2 Inheritance via Polymorphism (Painting Shapes)

Soal Praktikum
Silahkan membaca soal pada file lampiran: 2-Polymorphism via Inheritance.pdf
Source Code

Cylinder.java

```
public class Cylinder extends Shape {
   private double height, radius;

   public Cylinder( double radius, double height) {
       super("Cylinder");
       this.height = height;
       this.radius = radius;
   }

   @Override
   public double area() {
       return Math.PI * radius * radius * height;
   }

   @Override
   public String toString() {
        return super.toString() + " of height " + height + " and radius " +
radius;
   }
}
```

Paint.java

```
public class Paint
{
   private double coverage; //number of square feet per gallon
   //-----------------------------------------
   // Constructor: Sets up the paint object.
   //-----------------------------------------
   public Paint(double c)
   {
       coverage = c;
   }
   //-----------------------------------------------------
   // Returns the amount of paint (number of gallons)
   // needed to paint the shape given as the parameter.
   //-----------------------------------------------------

   public double amount(Shape s)
   {
       System.out.println ("Computing amount for " + s);
       return s.area() / coverage;
   }
}
```

PaintThings.java

```
import java.text.DecimalFormat;
public class PaintThings
{
   //-----------------------------------------
   // Creates some shapes and a Paint object
   // and prints the amount of paint needed
   // to paint each shape.
```

```
    //-----------------------------------------
    public static void main (String[] args)
    {
        final double COVERAGE = 350;
        Paint paint = new Paint(COVERAGE);
        Rectangle deck;
        Sphere bigBall;
        Cylinder tank;
        double deckAmt, ballAmt, tankAmt;

    // Instantiate the three shapes to paint
    // Compute the amount of paint needed for each shape
    // Print the amount of paint for each.

        deck = new Rectangle(20.0, 35.0);
        bigBall = new Sphere(15);
        tank = new Cylinder(10, 30);

        deckAmt = paint.amount(deck);
        ballAmt = paint.amount(bigBall);
        tankAmt =  paint.amount(tank);

        DecimalFormat fmt = new DecimalFormat("0.#");
        System.out.println ("\nNumber of gallons of paint needed...");
        System.out.println ("Deck " + fmt.format(deckAmt));
        System.out.println ("Big Ball " + fmt.format(ballAmt));
        System.out.println ("Tank " + fmt.format(tankAmt));
    }
}
```

Rectangle.java

```
public class Rectangle extends Shape {
    private double length, width;

    public Rectangle(double length, double width) {
        super("Rectangle");
        this.length = length;
        this.width = width;
    }
    @Override
    public double area() {
        return length * width;
    }

    @Override
    public String toString()
    {
         return super.toString() + " of length " + length + " and width " +
width;
    }
}
```

Shape.java

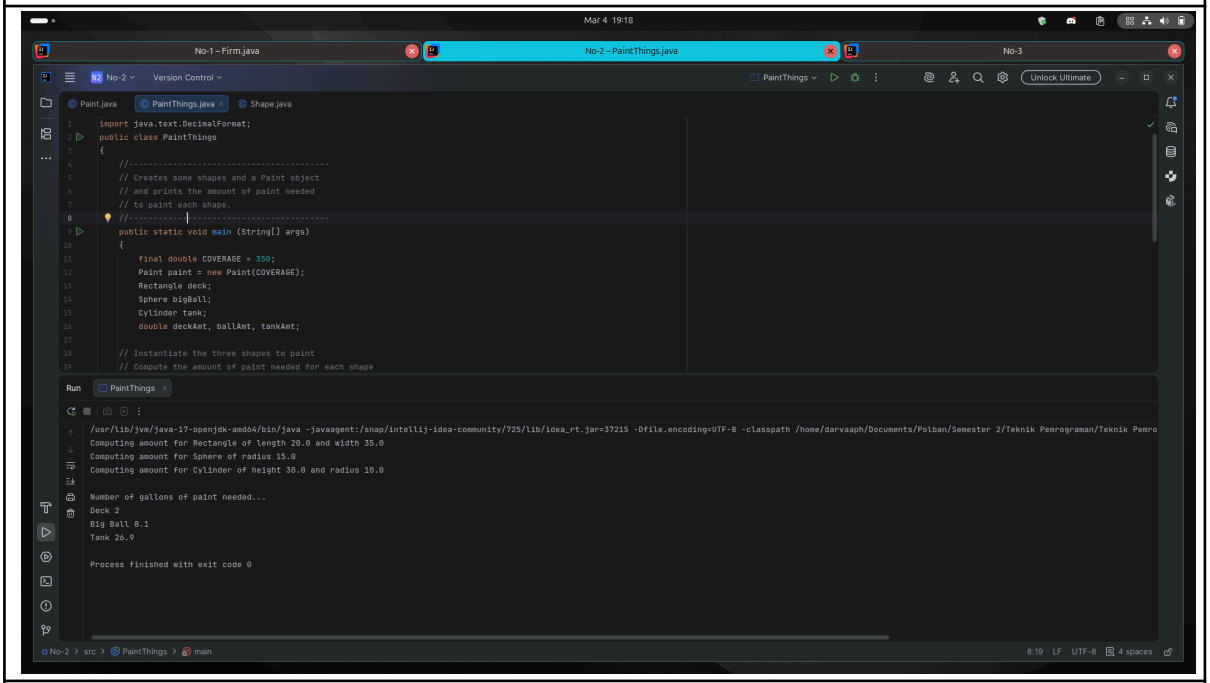
```
public abstract class Shape {
    String shapeName;

    public Shape(String shapeName){
        this.shapeName = shapeName;
    }

    abstract double area();
```

```
    public String toString() {
        return shapeName;
    }
}
```

Sphere.java

```
public class Sphere extends Shape
{
    private double radius; //radius in feet
    //-----------------------------------
    // Constructor: Sets up the sphere.
    //-----------------------------------
    public Sphere(double r)
    {
        super("Sphere");
        radius = r;
    }
    //-----------------------------------------
    // Returns the surface area of the sphere.
    //-----------------------------------------
    @Override
    public double area()
    {
        return 4*Math.PI*radius*radius;
    }
    //------------------------------------
    // Returns the sphere as a String.
    //------------------------------------
    @Override
    public String toString()
    {
        return super.toString() + " of radius " + radius;
    }
}
```

Screenshot Hasil



Penjelasan Permasalahan dan Solusi

Permasalahan:
Abstract class Shape belum ada, class Rectangle dan Cylinder belum dibuat, dan method amount() di Paint.java masih me-return 0 sehingga belum bisa menghitung jumlah cat.

Solusi (Shape.java): Membuat abstract class Shape dengan shapeName, constructor, area() dibuat abstract karena rumus luas tiap bentuk berbeda. toString() tidak perlu abstract karena cukup mengembalikan shapeName, yang sama untuk semua bentuk.

Solusi (Rectangle & Cylinder): Keduanya extends Shape dengan pola yang sama lalu buat constructor yang panggil super() dengan nama bentuknya, lalu override area() sesuai rumusnya masing-masing (length*width untuk Rectangle, Math.PI*r*r*h untuk Cylinder), dan override toString() untuk menampilkan dimensinya.

Solusi (Paint.java): Return statement diubah menjadi s.area() / coverage. s.area() akan otomatis memakai rumus yang sesuai tergantung bentuk yang dikirim dan inilah yang disebut dengan polymorphism."

Solusi (PaintThings.java): Tiga objek diinstantiasi deck = Rectangle(20,35), bigBall = Sphere(15), tank = Cylinder(10,30). Jumlah cat dihitung dengan paint.amount() lalu hasilnya dicetak.

Nama Teman Hal yang Dibantu (Opsional)

No. 3 Polymorphic Sorting

Soal Praktikum
Silahkan membaca soal pada file lampiran: 3-Polymorphic Sorting.pdf
Source Code

Numbers.java

```
import java.util.Scanner;
public class Numbers
{
   // ---------------------------------------------
   // Reads in an array of integers, sorts them,
   // then prints them in sorted order.
   // ---------------------------------------------
   public static void main (String[] args)
   {
       Integer[] intList;
       int size;
       Scanner scan = new Scanner(System.in);

       System.out.print ("\nHow many integers do you want to sort? ");
       size = scan.nextInt();
       intList = new Integer[size];

       System.out.println ("\nEnter the numbers...");

       for (int i = 0; i < size; i++)
           intList[i] = scan.nextInt();

       Sorting.insertionSort(intList);

       System.out.println ("\nYour numbers in sorted order...");

       for (int i = 0; i < size; i++)
           System.out.print(intList[i] + " ");

       System.out.println ();
   }
}
```

WeeklySales.java

```
import java.util.Scanner;

public class WeeklySales {
   public static void main(String[] args) {
       // deklarasi
       Salesperson[] salesList;
       int size;
       Scanner scan = new Scanner(System.in);

       // tanya jumlah salesnya
       System.out.print("\nEnter number of people: ");
       size = scan.nextInt();
       salesList = new Salesperson[size];

       System.out.println ("\nEnter the numbers...");

       // input data tiap sales person
       for (int i = 0; i < size; i++) {
           System.out.print("Enter first name: ");
           String firstName = scan.next();
```

```
            System.out.print("Last name: ");
            String lastName = scan.next();

            System.out.print("Enter total sales: ");
            int totalSales = scan.nextInt();

            salesList[i] = new Salesperson(firstName, lastName, totalSales);
        }


        // sort dan print
        Sorting.insertionSort(salesList);
        System.out.println ("\nRanking of Sales for the Week\n");
        for (Salesperson s : salesList)
            System.out.println (s);
    }
}
```

Salesperson.java

```
public class Salesperson implements Comparable
{
    private String firstName, lastName;
    private int totalSales;
    //-----------------------------------------------------
    // Constructor: Sets up the sales person object with
    // the given data.
    //-----------------------------------------------------
    public Salesperson (String first, String last, int sales)
    {
        firstName = first;
        lastName = last;
        totalSales = sales;
    }
    //-------------------------------------------
    // Returns the sales person as a string.
    //-------------------------------------------
    public String toString()
    {
        return lastName + ", " + firstName + ": \t" + totalSales;
    }
    //-------------------------------------------
    // Returns true if the sales people have
    // the same name.
    //-------------------------------------------
    public boolean equals (Object other)
    {
        return (lastName.equals(((Salesperson)other).getLastName()) &&
firstName.equals(((Salesperson)other).getFirstName()));
    }
    //--------------------------------------------------
    // Order is based on total sales with the name
    // (last, then first) breaking a tie.
    //--------------------------------------------------
    public int compareTo(Object other) {
        Salesperson otherPerson = (Salesperson)other; // casting

        if (totalSales < otherPerson.getSales()) {
            return -1;
        } else if (totalSales > otherPerson.getSales()) {
            return 1;
        } else {
```

```
            int lastNameCompare =
lastName.compareTo(otherPerson.getLastName());
            if (lastNameCompare != 0)
                return lastNameCompare;
            else
                return firstName.compareTo(otherPerson.getFirstName());
        }
    }
    //-------------------------
    // First name accessor.
    //-------------------------
    public String getFirstName()
    {
        return firstName;
    }
    //-------------------------
    // Last name accessor.
    //-------------------------
    public String getLastName()
    {
        return lastName;
    }
    //-------------------------
    // Total sales accessor.
    //-------------------------
    public int getSales()
    {
        return totalSales;
    }
}
```

Strings.java

```
import java.util.Scanner;

public class Strings {
    public static void main (String[] args) {
        String[] stringList;
        int size;
        Scanner scan = new Scanner(System.in);

        System.out.print ("\nHow many strings do you want to sort? ");

        size = scan.nextInt();
        stringList = new String[size];

        System.out.println ("\nEnter the strings...");

        for (int i = 0; i < size; i++)
            stringList[i] = scan.next();

        Sorting.insertionSort(stringList);

        System.out.println ("\nYour strings in sorted order...");

        for (int i = 0; i < size; i++)
            System.out.print(stringList[i] + " ");

        System.out.println ();
    }
}
```

Sorting.java

```java
public class Sorting
{
   //-----------------------------------------------------------------
// Sorts the specified array of objects using the selection
// sort algorithm.
//-----------------------------------------------------------------
   public static void selectionSort (Comparable[] list)
   {
       int min;
       Comparable temp;
       for (int index = 0; index < list.length-1; index++)
       {
           min = index;
           for (int scan = index+1; scan < list.length; scan++)
               if (list[scan].compareTo(list[min]) < 0)

           min = scan;
           // Swap the values
           temp = list[min];
           list[min] = list[index];
           list[index] = temp;
       }
   }
   //-----------------------------------------------------------------
   // Sorts the specified array of objects using the insertion
   // sort algorithm.
   //-----------------------------------------------------------------
   public static void insertionSort (Comparable[] list)
   {
       for (int index = 1; index < list.length; index++)
       {
           Comparable key = list[index];
           int position = index;
           // Shift larger values to the right
           while (position > 0 && key.compareTo(list[position-1]) > 0)
           {
               list[position] = list[position-1];
               position--;
           }
           list[position] = key;
       }
   }
}
```

Screenshot Hasil
Numbers.java

```java
import java.util.Scanner;
public class Numbers
{
    // ---------------------------------------------
    // Reads in an array of integers, sorts them,
    // then prints them in sorted order.
    // ---------------------------------------------
    public static void main (String[] args)
    {
        Integer[] intList;
        int size;
        Scanner scan = new Scanner(System.in);

        System.out.print ("\nHow many integers do you want to sort? ");
        size = scan.nextInt();
        intList = new Integer[size];

        System.out.println ("\nEnter the numbers...");
```

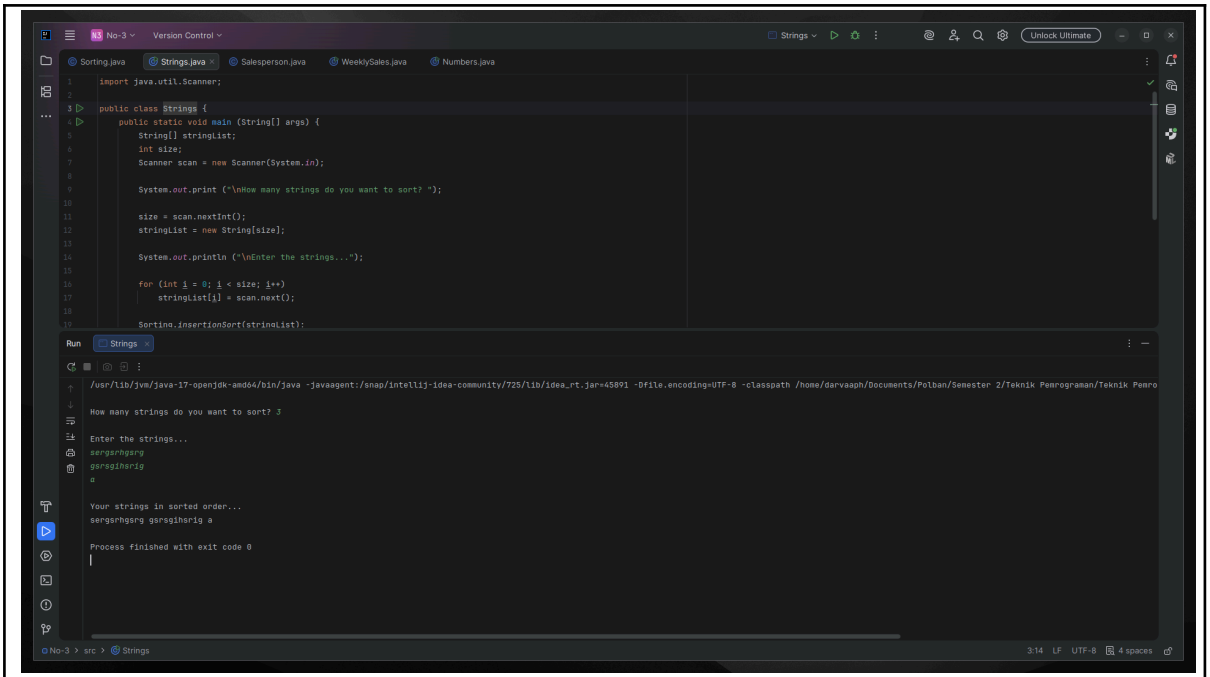
```
/usr/lib/jvm/java-17-openjdk-amd64/bin/java -javaagent:/snap/intellij-idea-community/725/lib/idea_rt.jar=37383 -Dfile.encoding=UTF-8 -classpath /home/darvaaph/Documents/Polban/Semester 2/Teknik Pemrograman/Teknik Pemro

How many integers do you want to sort? 4

Enter the numbers...
1
4134
14313413
2

Your numbers in sorted order...
14313413 4134 2 1

Process finished with exit code 0
```

WeeklySales.java

```java
import java.util.Scanner;

public class WeeklySales {
    public static void main(String[] args) {
        // deklarasi
        Salesperson[] salesList;
        int size;
        Scanner scan = new Scanner(System.in);

        // tanya jumlah salesnya
        System.out.print("\nEnter number of people: ");
        size = scan.nextInt();
        salesList = new Salesperson[size];

        System.out.println ("\nEnter the numbers...");

        // input data tiap sales person
        for (int i = 0; i < size; i++) {
            System.out.print("Enter first name: ");
```

```
Enter first name: Dodo
Last name: Modo
Enter total sales: 9992
Enter first name: Darva
Last name: Aryasatya
Enter total sales: 8882
Enter first name: Kucing
Last name: Hitam
Enter total sales: 22134

Ranking of Sales for the Week

Hermawan, Darva:    88883
Hitam, Kucing:  22134
Modo, Dodo:     9992
Aryasatya, Darva:   8882

Process finished with exit code 0
```

Strings.java



Penjelasan Permasalahan dan Solusi

Permasalahan A (Numbers.java):
Program tidak bisa dikompilasi karena menggunakan int[] sedangkan selectionSort() hanya menerima Comparable[]. Tipe primitif int bukan objek sehingga tidak bisa masuk.

Solusi A: Cukup ganti int[] menjadi Integer[]. Integer adalah versi objek dari int yang sudah bisa dibandingkan (Comparable). Input tidak perlu diubah karena autoboxing otomatis konversi int ke Integer.

Permasalahan B (Strings.java): Belum ada program untuk mengurutkan array of String.

Solusi B: Copy Numbers.java, ganti Integer[] ke String[], dan ganti scan.nextInt() ke scan.next(). String sudah implements Comparable jadi langsung bisa dipakai.

Permasalahan C (insertionSort): insertionSort() mengurutkan ascending, tapi yang dibutuhkan descending.

Solusi C: Ganti satu karakter saja: operator < 0 menjadi > 0 pada kondisi while. Ini membalik logika perbandingan sehingga elemen besar bergerak ke depan. Lalu update Numbers.java dan Strings.java untuk memanggil insertionSort().

Permasalahan D (compareTo()): Method compareTo() di Salesperson.java masih kosong sehingga sorting tidak bisa berjalan.

Solusi D: Melengkapi dengan logika bertingkat yaitu pertama tama kita cast Object ke Salesperson, bandingkan totalSales (return -1/1), jika sama bandingkan lastName, jika masih sama bandingkan firstName. Sesuai instruksi soal: sales sebagai kriteria utama, nama sebagai pemutus seri.

Permasalahan E (No. 7 Opsional - WeeklySales.java): Data salespeople masih hardcoded sehingga tidak fleksibel untuk diubah tanpa mengubah source code. **Solusi E:** Menambahkan Scanner untuk membaca jumlah salesperson dari user, lalu loop untuk input firstName, lastName, dan totalSales tiap orang. Ukuran array dibuat dinamis sesuai input. Data hardcoded dihapus seluruhnya.
Nama Teman Hal yang Dibantu (Opsional)